

1)Check for TL:DR ,ELI -15 in prompting and see the difference in output

Automating a web application with **Playwright** is highly efficient because it comes with built-in features like auto-waiting, cross-browser support (Chromium, Firefox, WebKit), and excellent debugging tools.

Here is a step-by-step guide to setting up and automating an application using Playwright with **Node.js/TypeScript** (the most popular stack for Playwright), followed by best practices.

---

## Step 1: Installation & Setup

First, create a new directory for your project, open your terminal inside it, and run the initialization command:

```
Bash
npm init playwright@latest
```

During setup, you will be prompted to choose:

- TypeScript or JavaScript (TypeScript is highly recommended).
- The name of your tests folder (default is tests).
- Whether to add a GitHub Actions workflow for CI/CD.
- Whether to install Playwright browsers (choose **Yes**).

### Project Structure

Once installed, your project will look like this:

- `playwright.config.ts`: The main configuration file where you define browsers, timeouts, and report formats.
  - `tests/`: The directory where you will write your automation scripts.
- 

## Step 2: Writing Your First Automation Script

Create a file named `login.spec.ts` inside the `tests/` folder. Here is a simple script that navigates to an application, fills out a login form, clicks submit, and verifies the successful login.

```
TypeScript
import { test, expect } from '@playwright/test';

test('User can successfully log into the application', async ({ page }) => {
  // 1. Navigate to the application URL
  await page.goto('https://example.com/login');
```

```
// 2. Interact with the elements using Locators
// Playwright automatically waits for these elements to become visible/actionable.
await page.getByPlaceholder('Enter your username').fill('testuser');
await page.getByLabel('Password').fill('SecurePassword123');

// 3. Click the submit button
await page.getByRole('button', { name: 'Log In' }).click();

// 4. Make an assertion (Web-First Assertions automatically retry until they pass)
await expect(page).toHaveURL('https://example.com/dashboard');
await expect(page.getByRole('heading', { name: 'Welcome back' })).toBeVisible();
});
```

---

## Step 3: Running Your Tests

You can execute your automation scripts using the command line:

- **Run all tests (Headless mode):** Runs tests in the background across all configured browsers.

Bash

```
npx playwright test
```

- **Run in UI Mode (Highly Recommended):** Opens a visual interface where you can see each test step, DOM snapshots, and execution logs.

Bash

```
npx playwright test --ui
```

- **Run a specific test file in Headed mode (Visible browser):**

Bash

```
npx playwright test login.spec.ts --headed
```

```
---
```

---

### ## Step 4: Viewing the Test Reports

Playwright generates an interactive HTML report automatically. If a test fails, it can show you exactly what went wrong.

To view the report after a run, execute:

```
```bash
```

```
npx playwright show-report
```

---

# Core Playwright Concepts to Master

To scale your automation beyond simple scripts, you need to leverage Playwright's core strengths:

## 1. Resilient Locators

Instead of relying on fragile CSS selectors or XPath expressions (e.g., `.btn-primary` or `/html/body/div...`), Playwright encourages locating elements the way a human interacts with them:

- `page.getByRole('button', { name: 'Submit' })`
- `page.getByText('Success!')`
- `page.getByPlaceholder('Search...')`

## 2. Built-in "Auto-Waiting"

Unlike older tools like Selenium, you almost never need to write manual sleep/wait commands (like `time.sleep()`). Playwright checks if an element is visible, enabled, and stable before performing clicks or typing actions.

## 3. Test Isolation & Auth Reuse

Every test runs in a completely isolated environment called a **Browser Context** (like a brand new incognito window). To avoid logging in before every single test, you can save your login state once and reuse it across your entire suite:

TypeScript

```
// Save auth state after logging in
await page.context().storageState({ path: 'auth.json' });
```

```
// Reuse it in your configuration or next test
test.use({ storageState: 'auth.json' });
```

---

## Secret Weapon: Test Generator (Codegen)

If you don't want to code everything from scratch, you can let Playwright write the script for you. Run the following command:

Bash

```
npx playwright codegen https://example.com
```

A browser window and a code inspector window will pop up. Every time you click, type, or interact with the website, Playwright will live-generate perfectly formatted TypeScript/JavaScript code that you can copy right into your test file.

## *Using TL;DR*

Here is the quick-start summary for automating an application with Playwright:

### 1. Initialize Project

Run this command in your terminal to set up Playwright, download browsers, and create a base configuration:

```
Bash
npm init playwright@latest
```

### 2. Write the Script

Create a test file (e.g., tests/login.spec.ts) using Playwright's human-centric locators and automatic retries:

```
TypeScript
import { test, expect } from '@playwright/test';

test('Quick Login Test', async ({ page }) => {
  await page.goto('https://example.com/login');
  await page.getByPlaceholder('Username').fill('myUser');
  await page.getByRole('button', { name: 'Log In' }).click();

  await expect(page).toHaveURL('https://example.com/dashboard');
});
```

### 3. Run and Debug

- **Run hidden (fastest):** `npx playwright test`
- **Run with UI (best for debugging):** `npx playwright test --ui`
- **Auto-generate test code by clicking around:** `npx playwright codegen`

---

**Why use it?** It features **auto-waiting** (no more flaky sleep commands), runs flawlessly across **Chromium, Firefox, and WebKit**, and isolates every test automatically so they don't interfere with one another.

## *ELI - 15*

Think of web automation like building a **robot companion** whose sole job is to test your website to make sure it isn't broken.

If you want your robot to test a website, you have to give it a recipe. Here is how you build and train your robot using **Playwright**:

---

## 1. Give the Robot a Brain (Installation)

First, you open your terminal (the text-based command center on your computer) and tell Playwright to install itself.

```
Bash
npm init playwright@latest
```

This downloads everything your robot needs to run, including its own secret versions of Google Chrome, Firefox, and Apple Safari.

## 2. Write the "Recipe" (The Code)

Next, you write a simple set of instructions. Because Playwright is smart, you can talk to it like a human. Instead of saying *"click on the blue box at coordinates X:200, Y:400,"* you just say *"click the button that says Log In."*

Here is what a recipe looks like:

```
TypeScript
import { test, expect } from '@playwright/test';

test('My Robot Test', async ({ page }) => {
  // 1. Go to the website
  await page.goto('https://example.com/login');

  // 2. Find the box that says "Username" and type into it
  await page.getByPlaceholder('Username').fill('CoolCoder123');

  // 3. Find the "Log In" button and click it
  await page.getByRole('button', { name: 'Log In' }).click();

  // 4. Double-check that it actually worked
  await expect(page).toHaveURL('https://example.com/dashboard');
});
```

## 3. Unleash the Robot (Running it)

Now you tell the robot to execute the recipe. You have two cool ways to do this:

- **The Ghost Mode (`npx playwright test`):** The robot runs the test in the background in less than a second. You won't see anything happen on your screen, but it will tell you "Success" or "Failed" when it's done.
  - **The Cyberpunk Mode (`npx playwright test --ui`):** This opens a futuristic dashboard. You can watch the robot click through the site, pause time, skip backward, and see exactly what the website looked like at every single millisecond.
-

## Why Playwright is like a Superpower:

- **It actually waits:** Old automation tools were dumb. If a website took 2 seconds to load, they would try to click a button that wasn't there yet and crash. Playwright automatically waits for the page to load and the button to appear before it clicks.
- **The Cheat Code (codegen):** If you run `npx playwright codegen`, it opens a browser window. You just click around the website like a normal person, and Playwright **automatically writes all the code for you** in the background.

### Using ELI-30 Test Architect

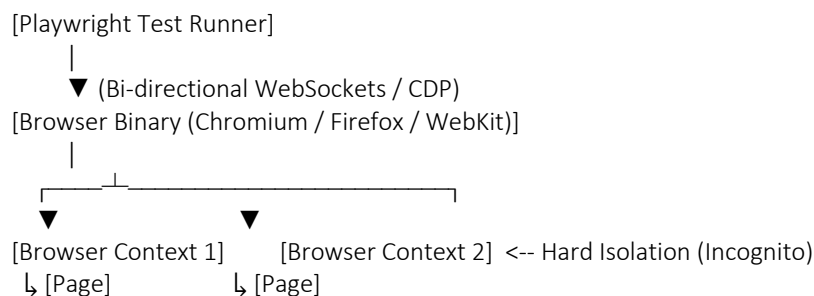
From a Test Architecture perspective, Playwright isn't just a browser automation driver; it's a modern, cohesive **test runner and orchestration platform** designed to address the systemic synchronization and flakiness issues inherent in WebDriver-based frameworks.

Here is how you architect and execute a scalable automation strategy using Playwright.

---

## 1. Architectural Foundation & Core Engine

Playwright bypasses the high-latency HTTP JSON wire protocols of legacy tools. Instead, it utilizes a single WebSocket connection to communicate directly with browser binaries via native debugging protocols (Chrome DevTools Protocol for Chromium, and equivalent low-level entry points for Firefox and WebKit).



### Key Architectural Advantages:

- **Zero-Overhead Isolation:** Playwright introduces the concept of **Browser Contexts**. Similar to an incognito window, a single browser instance can spin up hundreds of isolated contexts concurrently. Each context has its own cookies, local storage, and session state, eliminating state bleeding between parallel tests without the massive memory overhead of launching new browser instances.
- **The Flakiness Cure (Auto-Waiting):** Playwright fundamentally changes element interaction. Before performing any action (click, fill, hover), the engine runs a series of synchronous **Actionability Checks**. It guarantees the element is attached to the DOM, visible, stable (not animating), receiving events, and enabled.

- **Web-First Assertions:** Assertions like `expect(page.locator).toBeVisible()` are inherently asynchronous. They poll the DOM automatically up to a configurable timeout, eliminating explicit or implicit waits from your codebase.
- 

## 2. Scalable Framework Architecture (The Blueprint)

For an enterprise-grade solution, you want to decouple configuration, test data, page structures, and execution hooks.

### Project Structure

Plaintext

```
├─ playwright.config.ts    # Global engine, matrix, and environment config
├─ src/
│   ├─ pages/              # Page Object Models (POM)
│   │   └─ base.page.ts
│   │   └─ login.page.ts
│   └─ fixtures/           # Custom test fixtures for dependency injection
│       └─ app-fixtures.ts
└─ tests/                  # Clean declarative test specs
    └─ auth/
        └─ login.spec.ts
```

### High-Reusability Page Object Model (POM)

Using TypeScript, wrap your components and expose semantic actions rather than raw elements.

TypeScript

```
// src/pages/login.page.ts
import { Locator, Page } from '@playwright/test';

export class LoginPage {
  private readonly usernameInput: Locator;
  private readonly passwordInput: Locator;
  private readonly submitButton: Locator;

  constructor(private readonly page: Page) {
    // Locators are lazy-evaluated; declaring them here doesn't query the DOM yet
    this.usernameInput = page.getByPlaceholder('Username');
    this.passwordInput = page.getByLabel('Password');
    this.submitButton = page.getByRole('button', { name: 'Log In' });
  }

  async navigate() {
    await this.page.goto('/login'); // Base URL is injected via config
  }

  async login(user: string, pass: string) {
    await this.usernameInput.fill(user);
```

```
    await this.passwordInput.fill(pass);
    await this.submitButton.click();
  }
}
```

---

### 3. Global Configuration & Multi-Browser Matrix

The playwright.config.ts file handles your execution strategy, parallelization parameters, and environment matrixing.

#### TypeScript

```
// playwright.config.ts
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './tests',
  fullyParallel: true, // Run tests within a single file concurrently
  workers: process.env.CI ? 4 : '50%', // Scales workers based on environment
  retries: process.env.CI ? 2 : 0, // Defers flakiness reporting to a retry bucket
  reporter: [['html'], ['list']],

  use: {
    baseURL: 'https://api.prd.example.com',
    trace: 'on-first-retry', // Generates rich debug zip files on failure
    screenshot: 'only-on-failure',
  },

  /* Configure projects for major browser engines */
  projects: [
    {
      name: 'chromium',
      use: { ...devices['Desktop Chrome'] },
    },
    {
      name: 'firefox',
      use: { ...devices['Desktop Firefox'] },
    },
    {
      name: 'webkit',
      use: { ...devices['Desktop Safari'] },
    },
  ],
});
```

---

### 4. Advanced Architecture Patterns to Enforce

As a Test Architect, ensure your team leverages Playwright's native optimization hooks instead of porting over legacy paradigms:

#### Global Authentication Reuse (The 10x Speedup)

Don't login via the UI before every test. Authenticate once during a global setup phase, export the storage state (cookies and localStorage), and inject it directly into browser contexts.

#### TypeScript

```
// Enforce via config or a setup project
import { test as setup } from '@playwright/test';

setup('authenticate user', async ({ page }) => {
  await page.goto('https://example.com/login');
  // ... login flow ...
  await page.context().storageState({ path: 'storage/user.json' });
});
```

## Network Interception & Mocking

Playwright lets you intercept, modify, or mock API traffic directly at the browser context level. This allows for excellent deterministic UI testing without a brittle backend dependency.

#### TypeScript

```
// Mock a slow network request or an unready API route
await page.route('**/api/v1/analytics', async (route) => {
  await route.fulfill({
    status: 200,
    contentType: 'application/json',
    body: JSON.stringify({ metrics: [10, 20, 30] }),
  });
});
```

## UI Mode & The Trace Viewer

Eliminate manual log parsing. Playwright's Trace Viewer (`npx playwright show-trace`) captures a full zip archive of execution. It gives engineers a time-traveling DOM snapshot, network logs, console outputs, and action source code mapping for every single step executed on your CI pipeline.

## 2. Chain of Thoughts

Generate Automation Framework Architecture

Act as an Automation Architect to design the framework for Playwright Automation with Python

#### Instructions

[MANDATORY] Ask me required multiple choice questions one by one  
[SHALL] consider environment as Oracle HRMS  
[MANDATORY] Once after answering the first question ,proceed with next  
[MANDATORY] Generate Framework Structure FirstAfter completing all the questions  
[STRICT] Proceed Step by Step for framework implementation  
[STRICT] Up on my approval proceed with the next step  
[MANDATORY] Mention Prepared by Azmat with designation at the end of document  
[CRITICAL] Once after approving all steps generate Framework document in pdf format and include Framework structure and step by step

#### Tone

Generate in Professional way

[https://github.com/azmatrana21/genai-assignments/blob/main/phase-1/week-2/day-1/Oracle\\_HRMS\\_Playwright\\_Architecture\\_Framework.pdf](https://github.com/azmatrana21/genai-assignments/blob/main/phase-1/week-2/day-1/Oracle_HRMS_Playwright_Architecture_Framework.pdf)

### ***Generate Test strategy document***

#### Context

Act as a Experienced Oracle HRMS Domain Test Manager

#### Instruction

Design a Test strategy document to test the feature where an Organization is consolidating its disparate, legacy regional payroll and HR systems into a unified Oracle Cloud HCM (or Oracle EBS R12.2) platform. The initial footprint covers the US, UK, and India, affecting over 45,000 employees.

[MANDATORY] Use JIRA As test Management Tool

[MANDATORY] Ask me 5 valid multiple choice questions one by one

[STRICT] After answering one question Proceed with next question

[CRITICAL] After answering all questions generate Test strategy document

[HIGHPRIORITY] Stick to the output format

#### Persona

Act as Oracle HRMS SME to review the test strategy document and generate score

#### Output

Generate the Test Strategy Document as downloadable pdf

[https://github.com/azmatrana21/genai-assignments/blob/main/phase-1/week-2/day-1/Oracle\\_HRMS\\_Global\\_Consolidation\\_Test\\_Strategy.pdf](https://github.com/azmatrana21/genai-assignments/blob/main/phase-1/week-2/day-1/Oracle_HRMS_Global_Consolidation_Test_Strategy.pdf)